

Project 2 - Reliable Transport

Yale CPSC 433/533 Computer Networks, Spring 2022

Version 1.1

Due: March 30, 2022 at 11:59 pm

In this project, you will build a simple **reliable transport protocol** called “BULLDOGS-TP” (BTP). Your protocol must provide **in-order, reliable delivery of UDP datagrams**, and must do so in the presence of packet loss, delay, corruption, duplication, and re-ordering.

We will provide you with a reference implementation of a receiver (which you must use) that returns a cumulative ACK whenever it receives a data packet. Your job is to **implement a sender** that, when sending packets to this receiver, achieves reliable delivery.

Protocol Description

Our simple protocol has four message types: `start`, `end`, `data`, and `ack`.

`start`, `end`, and `data` messages all follow the same general format:

```
start|<sequence number>|<data>|<checksum>
data|<sequence number>|<data>|<checksum>
end|<sequence number>|<data>|<checksum>
```

- The first data packet in a connection will be a `start` message with a starting sequence number (of your choice). The receiver will use the sequence number provided as the initial sequence number for all packets in that connection.
- After sending the `start` message, send additional packets in the same connection using the `data` message type (incrementing the sequence number appropriately)
- Unsurprisingly, the last data in a connection should be transmitted with the `end` message type to signal the receiver that the connection is complete.

Your sender should accept acknowledgments from the receiver in the format:

```
ack|<sequence number>|<checksum>
```

An important limitation is the **maximum size of your packets**. The UDP protocol has an 8-byte header, and the IP protocol underneath it has a header of ~20 bytes. Because we will be using Ethernet networks, which have a **maximum frame size of 1500 bytes**, this leaves 1472 bytes for your entire packet (message type, sequence number, data, and checksum).

On the other hand, small packet size hurts performance. Therefore, in order to meet the performance requirement, packet sizes must be larger than 1000 bytes but less than 1472 bytes.

Note that the `end` packet in a connection is permitted to have fewer than 1000 bytes of data since it must take whatever residual data is left in the stream. Another exception occurs if the payload being sent is shorter than 1000 bytes. In this case, you can send the entire payload in the `start` packet followed by an `end` packet with no data.

Note that the angle brackets ("`<`" and "`>`") are not part of the protocol, but the "`|`" delimiters are. You should ensure that there are no extra spaces between your delimiters ("`|`" character) and the fields of your packet. For specific formatting details, see the provided sample code.

Receiver specification

The simple receiver we will provide you with will be used for grading, so make sure that your sender is compatible with it.

The BULLDOGS-TP receiver responds to data packets with **cumulative acknowledgments**. Upon receiving a message of type `start`, `data`, or `end`, the receiver generates an `ack` message with the sequence number it expects to receive next, which is the lowest sequence number not yet received. In other words, if it expects a packet of sequence number N , the following two scenarios may occur

1. If it receives a packet with a sequence number not equal to N , it will send "`ack|N`".
2. If it receives a packet with sequence number N , it will check for the highest sequence number (say M) of the in-order packets it has already received and it will send "`ack|M+1`". For example, if it has already received packets $N+1$ and $N+2$ (i.e. $M = N+2$), but no others past $N+2$, then it will send "`ack|N+3`".

Let's illustrate this with an example. Suppose packets 0, 1, and 2 are sent, but packet 1 is lost before reaching the receiver. The receiver will send "`ack|1`" upon receiving packet 0, and then "`ack|1`" again upon receiving packet 2. As soon as the receiver receives packet 1 (due to retransmission from the sender), it will send "`ack|3`" (as it has already received packet 2, and upon receiving this acknowledgement the sender can assume all three packets were successfully received).

If the next expected packet is N , the receiver will drop all packets with sequence number greater than $N+4$; that is, the receiver operates with a window of five packets, and drops all packets that fall outside of that range. When the next unexpected packet is $N+1$ (due to N arriving), then the receiver will accept packet $N+5$.

You can assume that once a packet has been acknowledged by the receiver, it has been properly received. The receiver has a default timeout of 10 seconds; it will automatically close any connections for which it does not receive packets for that duration.

Sender specification

The sender should read an input file and transmit it to a specified receiver using UDP sockets. It should split the input file into appropriately sized chunks of data, specify an initial sequence number for the connection, and append a checksum to each packet. The sequence number should increase by one for each additional packet in a connection. Functions for generating and validating packet checksums will be provided for you (see `Checksum.py`).

Your sender must implement a reliable transport algorithm based on the “Go Back N” (or GBN) protocol. Remember that the **receiver’s window size is five packets**, and any packets it receives outside this window will be ignored. Your sender must be able to accept `ack` packets from the receiver. Any `ack` packets with an invalid checksum should be ignored.

Your sender should provide reliable service under the following network conditions:

- **Loss:** arbitrary levels; you should be able to handle periods of 100% packet loss, i.e., large sequences of packet loss
- **Corruption:** arbitrary types and frequency
 - Your Sender should drop/disregard invalid packets it receives from the Receiver
 - Some examples of corruption that may occur are incorrect packet format, incorrect checksum, and incorrect or impossible ACK numbers
- **Re-ordering:** i.e. packets may arrive in any order
- **Duplication:** you could see a packet any number of times.
- **Delay:** packets may be delayed indefinitely (but in practice, generally not more than 10s)

We will test your senders in a variety of scenarios: e.g., packet loss or data corruption. Just because your sender is compatible with the receiver in one test environment does not ensure a perfect score! Your sender must be capable of handling all scenarios outlined in this document.

Your sender should be invoked with the following command:

```
python3 Sender.py -f <input file>
```

Additional Sender Notes

- The sender should implement a **500ms retransmission timer** to automatically retransmit packets that were never acknowledged (potentially due to `ack` packets being lost). We do not expect you to use an adaptive timeout. This can be done using the “time” module in Python.
- Your sender should support a **window size of 5 packets** (i.e., 5 unacknowledged packets).
- Your sender should roughly meet or exceed the performance (in both time and number of

packets required to complete a transfer) of a properly implemented Go Back N based BULLDOGS-TP sender.

- Your sender should be able to handle arbitrary message data (i.e., it should be able to send an image file just as easily as a text file).
- Any packets received with an invalid checksum should be ignored.
- If you get an error “module psutil not found” please install psutil using pip (for Python3) by running:
 - `pip3 install psutil -user`
- You cannot use any other external Python packages besides the standard modules (you should not need anything besides `time`, `base64`, and `psutil`)
- Make sure your implementation runs on Zoo machines using Python 3. We do not anticipate that the version of Python 3 that you use will affect your implementation’s correctness, but we will use version 3.9.7 for grading submissions.
- Your sender should be written in **Python3** and adhere to the standards given in [PEP 8](#), Style Guidelines for Python Code
- Your sender **must not produce console output** during normal execution
 - Python exception messages are ok but try to avoid having your program crash in the first place.
 - As long as the error does not cause your sender to crash, do not print to stdout nor stderr
- A connection can be considered completed if and only if the end packet has been acknowledged by the receiver
- A connection must always, **at minimum**, have a `start` and `end` packet regardless of how small the file is—even if the file is empty. **Hint: for very small packets, it is usually easier to send all data in the `start` packet followed by an empty `end` packet.**
- In Python 3, data read from files takes the form of a bit stream rather than a string. Likewise, data needs to be in the form of a bit stream when you send it to the receiver. To manipulate the data from a file before sending it to the receiver, you will need to convert it into a string first. **If you import `base64`, you can use commands of the following form to convert a bit stream into a string, where `DATA_READ_FROM_A_FILE` is the bit stream from a file:**
 - `data = base64.b64encode(DATA_READ_FROM_A_FILE)`
 - `decoded_data = data.decode()`
- The `send` method in `BasicSender.py` takes care of converting strings back into bit streams. You do not need to convert the strings back into bit streams yourself before sending them.

We will evaluate your sender on correctness, time of completion for a transfer, and the number of packets sent (and re-sent). Transfer completion time and the number of packets used in a transfer will be measured against our own reference implementation of this “Go Back N”-based sender. Note that a “Stop-And-Wait” sender [described below] will not have adequate performance.

Performance Improvements

Correctly implementing a simple Go-Back-N transport protocol will give you **85%** of the total score. You can add the following performance enhancements, for the remaining 15% credits.

Fast Retransmit (5%): Go Back N sender retransmits the unacknowledged packet only on a timeout event, which can wait up to 500ms. Instead, we can detect the packet loss event by duplicated acknowledgements (DupACK) event. If we receive three DupACKs (that is, receiving the same acknowledgement number 4 times), we are almost sure the packet has been lost and we should immediately retransmit the packet.

For example, if the sender sends five packets with sequence numbers 0,1,2,3,4, but packet 1 is lost. The receiver will send cumulative acknowledgment `ack|1` four times back to the sender (upon receiving each of the packets 0,2,3, and 4). Upon receiving the fourth `ack|1`, the sender should retransmit packet 1.

At the point of retransmission, the sender should reset the timeout timer and start counting DupACKs back from 0.

Selective Acknowledgments (10%): If the sender knows exactly what packets the receiver has received; it can retransmit only the missing packets (rather than naively retransmitting the last N packets). To earn these 10 points, you should implement SACK, or **Selective Acknowledgment**, functionalities at the sender.

We have modified the receiver so that it can provide explicit acknowledgment of which packets have been received. The Receiver.py runs in SACK mode if you enable the `-k` or `--sack` option. In SACK mode, in addition to sending a cumulative acknowledgment, the receiver also sends selective acknowledgments in the following format.

```
sack|<cum_ack;sack1,sack2,sack3,...>|<checksum>
```

`cum_ack` is the cumulative acknowledgement number, which is the same as the one in an ack packet. Following the `cum_ack` are a *semicolon* and a list of `sack`'s separated by a *comma*. For example, if the sender sends five packets with sequence numbers 0,1,2,3,4, but packets 1 and 2 are lost. The following table shows the sack packets sent by the receiver when a packet is received.

Packet Received	sack packet sent by the receiver
0	<code>sack 1; <checksum></code>
3	<code>sack 1;3 <checksum></code>

4	sack 1;3,4 <checksum>
---	-----------------------

On getting a timeout, the sender should therefore retransmit only packets with sequence numbers 1 and 2.

If you have implemented Fast Retransmit, when a third duplicated acknowledgement is seen, the sender should **retransmit all non-ACKed packets**, not just the first one in the window.

The sender should be running in SACK mode when the -k or --sack option is on. We test your sender by running:

Receiver side: `python3 Receiver.py -k`

Sender side: `python3 Sender.py -f <filename> -k`

In SACK mode, your sender should only retransmit packets that are missing. For instance, in the previous example that packet 1 is missing, the sender should only retransmit packet 1 after the timeout.

Project Specification (see README file for details)

Receiver.py is the sample receiver. The version of Receiver.py here is largely identical to what we will use for grading your projects (what we actually use may include additional instrumentation to assist in grading, but should not vary significantly in functionality). Feel free to modify it, but keep in mind that we will be testing against our own version of the sender, not yours. If bugs are found or changes made to the sample receiver, we'll notify you and post an updated copy.

The BasicSender class in BasicSender.py provides a skeleton upon which to build your reliable sender. It provides the following methods:

`__init__(self, dest, port, filename, debug=False, sackMode=False)`: Creates a BasicSender. Specify the destination's hostname, the port at which the receiver is listening, and a filename to transmit. If no filename is provided, it will read from STDIN.

`receive(self, timeout)`: Receive a packet. Waits for a packet before returning. Optionally you can specify a maximum timeout to wait for a packet. Returns the received packet as a string, or None if receive times out.

`send(self, message)`: Sends message to the receiver specified when you created the sender.

`make_packet(self,msg_type,seqno,message)`: Creates a BULLDOGS-TP packet from the specified message type, sequence number, and message. Generates the appropriate checksum, and returns the full BULLDOGS-TP packet with checksum appended.

`split_packet(self,packet)`: Given a BULLDOGS-TP packet, splits a packet into a tuple of the form (msg_type, seqno, data, checksum). For packets without a data field, the data element will be the empty string.

In addition, it defines one method which you must implement:

`start(self)`: Starts the Sender.

We provide three example senders that build upon the BasicSender; you are welcome to base your design upon these:

UnreliableSender.py: While it provides no reliability, it will read from a file or STDIN and send to a receiver using our protocol.

InteractiveSender.py: A simple interactive sender. It will send any message you type to the specified receiver. It then waits for a response before prompting you for a new message.

HarvurdSender.py: Almost identical to InteractiveSender.py, it's not very good at counting and sometimes loses track of sequence numbers. You can use this to see how the receiver behaves when loss occurs.

Checksum.py includes two functions for validating and generating checksums for your packets:

`validate_checksum(message)`: Returns true if the message's checksum matches the message, and false otherwise. This function assumes the last field of the message the checksum.

`generate_checksum(message)`: Returns the checksum string for a message. This function assumes the message includes the trailing delimiter. The checksum is **ONLY** valid if you simply append this function's result to the message you pass in.

Testing

You are responsible for testing your own project, that is, you are expected to write test cases for your own code to ensure compliance with the project specifications. In order to make this process easier for you, we've included a test harness that you can use to develop tests for your project. The test harness is designed to intercept all packets sent between your sender and the receiver. It can modify the stream of packets and check to ensure the stream meets certain conditions. This is very similar to

the grading script that we will use to evaluate your projects. We have provided three public test cases (BasicTest, RandomDropTest, and SackRandomDropTest) as examples of how to use the test harness. These test cases send a README file using the specified sender implementation to the specified receiver implementation, either passing all packets through the forwarder unmodified or dropping random packets. They both then verify that the file received by the receiver matches the input.

To run a test using this test harness, do the following:

```
python3 TestHarness.py -s YourSender.py -r Receiver.py
```

where "YourSender.py" is the path to your sender implementation, "Receiver.py" is the path to the receiver implementation. Inside TestHarness.py, you need to modify the function "tests_to_run" at the top of the file to include any test cases you want.

Note that **passing the example test cases is a necessary but not sufficient condition for a good score** on this project! These are only meant as tests of basic functionality and examples for your own test cases; you must think about the types of edge cases your sender could encounter, as well as the various conditions required for correctness.

Hints and Tips

To begin with, just focus on the simple case where nothing bad ever happens to your packets. After you have that case working, you can consider how to handle packet loss.

It may help to build your way up to a full-fledged reliable sender. The simplest reliable transport mechanism is "Stop-And-Wait", in which the sender transmits a single packet and waits for the receiver to acknowledge its receipt before transmitting more. You could start by building a "Stop-And-Wait" sender and extending that for the full Go Back N based sender. Understand that the "Stop-And-Wait" sender is only a stepping stone, and we require you to submit a full BULLDOGS-TP sender for evaluation.

Submissions

Check the lecture schedule page on Canvas for the latest copy of the project code.

Submit a tar file with the following format for the tar file:

```
project2-lastname-firstname.tar
```

The tar file should contain only the following two files (case sensitive):

- Sender.py
- README.txt

The README.txt file you submit with your code should contain:

- You name and NetID
- How challenging was this project on a scale of 1 (easiest) to 10 (hardest)?
- How long did it take you?

Your implementation should be entirely in `Sender.py` because the auto grader only copies this file to the grading directory. So, if you want to define any new class, try putting it in `Sender.py`.

Make sure your tar file passes the format check script, which should be run in the base bulldogs-tp directory with all other supporting files. You can run the following to create a tar file and then run check the format of the file:

```
$ tar -cf project2-lastname-firstname.tar Sender.py README.txt $
./format-check.sh project2-lastname-firstname.tar
```

The expected output is:

```
Found Sender.py
Found README.txt
Now running 'BasicTest'...
Test passes!
Now running 'RandomDropTest'...
Test passes!
```

If this is what you get, then submit the tar file on Canvas. If you get permission denied for running `format-check`, you need to first run

```
$ chmod +x format-check.sh
```

Collaboration Policy

You may not share code with any classmates, including your tests. You may discuss the assignment requirements or your solutions (e.g., what data structures were used) - *away from a computer and without sharing code* - but you should not discuss the detailed nature of your solution (e.g., what algorithm was used to compute checksums). Also, don't put your code in a public repository. Assignments suspected of cheating or forgery will be handled according to Yale's undergraduate and/or graduate regulations.